

Routing with React Router

React shows one page — so how do you build many? Learn why routing isn't built in, and use React Router for multiple pages, shared layouts, and dynamic URLs.

WHAT WE COVER

[Why routing](#)[Routes & Links](#)[Outlet layouts](#)[Dynamic routes](#)[useParams](#)[Query params](#)

So far your React apps live on a single page. Real apps have many — a home page, a products list, a product detail page, an about page — each with its own URL you can bookmark and share. React doesn't do this on its own, so we use React Router. Today you'll learn why routing isn't built in, set up multiple pages, share a layout with Outlet, and build dynamic URLs with useParams and query params.

Routing with React Router

A **Single-Page App** loads once and swaps content with JavaScript. **React Router** maps each URL to a component, so you get real, shareable pages and back/forward buttons that work — without full page reloads.

CLASS AGENDA • ~2 HOURS

0:00	Why React has no built-in routing	\$1
0:15	Install React Router & first routes	\$2-3
0:35	Navigating with Link & NavLink	\$4
0:55	Shared layouts with nested routes & Outlet	\$5
1:15	Dynamic routes, useParams & query params	\$6-7
1:40	Programmatic nav, 404 & project	\$8-10

ON THIS HANDOUT

- | | |
|--|--|
| 01. Why React has no built-in routing | 06. Dynamic routes & useParams |
| 02. What React Router does | 07. Query params with useSearchParams |
| 03. Setting up routes | 08. Programmatic navigation |
| 04. Navigating: Link & NavLink | 09. The 404 catch-all route |
| 05. Nested routes & Outlet layouts | 10. Project: a multi-page app |

PART 1 • Why Routing?

01 Why React has no built-in routing

React is a **UI library** — its job is turning data into components. It deliberately leaves bigger app concerns (like routing) to separate tools, so you can choose what fits. That's why there's no `<Route>` in React itself.

There's also a deeper reason. A React app is a **Single-Page Application (SPA)**: the browser loads one HTML page, and JavaScript swaps the content. But normally, changing the URL makes the browser **reload** the whole page from the server — which throws away your app and its state.

- ▶ We want the URL to change (so pages are bookmarkable and the back button works)...
- ▶ ...but **without** a full reload that restarts the whole app.
- ▶ That trick — swapping views based on the URL, client-side — is called **client-side routing**.

React Router is the standard library that provides it.

02 What React Router does

React Router watches the URL and renders the component you've mapped to it — no server round-trip, no reload. Install it into your Vite project (same npm world as always):

```
npm install react-router-dom
```

TERMINAL

- ▶ It maps each URL path to a component (a “route”).
- ▶ It updates the browser URL and history so back/forward work.
- ▶ It swaps the visible component instantly, keeping the app alive.

PART 2 · Core Routing

03 Setting up routes

Wrap your app in `<BrowserRouter>`, then list your pages with `<Routes>` and `<Route>`. Each route maps a `path` to an `element`.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function Home() { return <h1>Home</h1>; }
function About() { return <h1>About</h1>; }
function Products() { return <h1>Products</h1>; }

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/products" element={<Products />} />
      </Routes>
    </BrowserRouter>
  );
}
```

TSX

- ▶ `<BrowserRouter>` enables routing — wrap it around everything, once.
- ▶ Each `<Route>` says: when the URL is this `path`, render this `element`.
- ▶ Visiting `/about` now shows the About page — no reload.

04 Navigating: Link & NavLink

Never use a plain `<a href>` for internal links — it reloads the whole page. Use `<Link>` (or `<NavLink>`) to navigate client-side, instantly.

```
import { Link, NavLink } from "react-router-dom";  
  
function Navbar() {  
  return (  
    <nav>  
      <Link to="/">Home</Link>  
      <Link to="/products">Products</Link>  
  
      {/* NavLink knows when it is the active route -> style it */}  
      <NavLink to="/about" className={({ isActive }) => (isActive ? "active" : "")}>  
        About  
      </NavLink>  
    </nav>  
  );  
}
```

TSX

- ▶ `<Link to="/path">` navigates without reloading the app.
- ▶ `<NavLink>` adds an `isActive` flag so you can highlight the current page.

05 Nested routes & Outlet layouts

Most sites share a layout — the same navbar and footer on every page. Instead of repeating it, make a **layout route** and render child routes inside it with `<Outlet />`.

```
import { Routes, Route, Outlet, Link } from "react-router-dom";

function Layout() {
  return (
    <div>
      <nav><Link to="/">Home</Link> <Link to="/products">Products</Link></nav>
      <main>
        <Outlet />      {/* the matched child route renders HERE */}
      </main>
      <footer>(c) 2026</footer>
    </div>
  );
}

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />} > {/* parent = layout */}
      <Route index element={<Home />} />      {/* "/" */}
      <Route path="products" element={<Products />} />  {/* "/products" */}
      <Route path="about" element={<About />} />        {/* "/about" */}
    </Route>
  </Routes>
  );
}
```

- ▶ The parent route renders the shared layout; `<Outlet />` is where each child page appears.
- ▶ `index` is the default child shown at the parent's exact path (`/`).
- ▶ The navbar and footer are written once and appear on every page.

PART 3 • Dynamic & Query URLs

06 Dynamic routes & useParams

You can't write a route for every product. Instead, use a **dynamic segment** with a colon — `:id` — and read the value with `useParams`.

```

// route definition -- :id is a placeholder
<Route path="products/:id" element={<ProductDetail />} />

// inside the component
import { useParams } from "react-router-dom";

function ProductDetail() {
  const { id } = useParams<{ id: string }>(); // read the :id from the URL

  return <h1>Showing product #{id}</h1>;
}

// visiting /products/42 -> id === "42"

```

TSX

- ▶ `path="products/:id"` matches `/products/1`, `/products/42`, and so on.
- ▶ `useParams()` returns the URL parameters; here `id` is the part after `/products/`.
- ▶ This is how detail pages, user profiles, and blog posts work.

07 Query params with useSearchParams

Query params are the `?key=value` part of a URL — great for filters, search terms and sorting (e.g. `/products?category=shoes&sort=price`). Read and update them with `useSearchParams`.

```

import { useSearchParams } from "react-router-dom";

function ProductList() {
  const [params, setParams] = useSearchParams();
  const category = params.get("category") ?? "all"; // read ?category=

  return (
    <div>
      <p>Category: {category}</p>
      <button onClick={() => setParams({ category: "shoes" })}>Shoes</button>
      <button onClick={() => setParams({ category: "bags" })}>Bags</button>
    </div>
  );
}

```

TSX

- ▶ `params.get("category")` reads a query value; `setParams(...)` updates the URL.
- ▶ Query params keep filters in the URL, so a filtered view can be bookmarked and shared.

A note on naming: React Router doesn't ship a hook literally called `useQueryParams` — `useSearchParams` is the built-in one for query strings. Teams sometimes wrap it in their own small helper hook and call it `useQueryParams`:

```
// a thin custom hook on top of useSearchParams
function useQueryParam(key: string): string | null {
  const [params] = useSearchParams();
  return params.get(key);
}

const category = useQueryParam("category");
```

TSX

08 Programmatic navigation

Sometimes you navigate from code, not a click — after a form submits, or a login succeeds. Use the `useNavigate` hook.

```
import { useNavigate } from "react-router-dom";

function LoginButton() {
  const navigate = useNavigate();

  function handleLogin() {
    // ...do the login...
    navigate("/dashboard"); // go to another route from code
  }

  return <button onClick={handleLogin}>Log in</button>;
}

// navigate(-1) goes back one page, like the browser back button
```

TSX

- ▶ `navigate("/path")` moves to a route; `navigate(-1)` goes back.
- ▶ Use it after an action completes (form submit, login, delete).

09 The 404 catch-all route

Add a route with `path="*"` at the end to catch any URL that didn't match — your “page not found” screen.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/products" element={<Products />} />
  <Route path="*" element={<NotFound />} />  {/* matches anything else */}
</Routes>
```

TSX

- * matches whatever the earlier routes didn't, so an unknown URL shows a friendly 404 instead of a blank screen.

PART 4 • Project

10 Project: a multi-page app

Put it together into a small shop with real pages and a shared layout.

The route map

```

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Layout />}>
      <Route index element={<Home />} />
      <Route path="products" element={<ProductList />} />
      <Route path="products/:id" element={<ProductDetail />} />
      <Route path="about" element={<About />} />
      <Route path="*" element={<NotFound />} />
    </Route>
  </Routes>
</BrowserRouter>

```

TSX

How the pieces connect

- ▶ `Layout` holds the navbar + footer and an `<Outlet />` for each page.
- ▶ `ProductList` shows cards; each links to `/products/:id` via `<Link>`.
- ▶ `ProductDetail` reads the `:id` with `useParams` and shows that product.
- ▶ A filter on the list uses `useSearchParams` so `/products?category=shoes` is shareable.
- ▶ A `*` route shows a 404 for anything else.

This is the exact structure you'll reuse for the e-commerce front-end later in the roadmap — every page reachable by its own URL.

TRY IT YOURSELF

Build the four pages above with a shared layout. Add a “Back to products” button on the detail page using `useNavigate(-1)`.

Practice Challenges

Cement routing with a few of these:

- ▶ Add an `/about` and a `/contact` page to the shared layout.
- ▶ Give the navbar active-link styling with `NavLink` and `isActive`.
- ▶ Make a `/users/:username` route that greets the user by name from `useParams`.
- ▶ Add a sort dropdown that writes `?sort=price` with `useSearchParams`.
- ▶ After a fake “Add to cart”, use `useNavigate` to send the user to `/cart`.

Conclusion & what's next

You can now turn a single-page React app into a real multi-page experience: you know **why** routing lives in a separate library, how to define routes, navigate with **Link/NavLink**, share layouts with **Outlet**, build dynamic URLs with **useParams**, handle query strings with **useSearchParams**, and navigate from code with **useNavigate**.

Next class (Day 13): building forms in React — controlled inputs, multiple fields, submission and validation — the last core skill before we assemble reusable components and a full e-commerce front-end.

QUICK RECAP

React has no built-in routing (it's a UI library + SPAs avoid reloads) · **React Router** maps URLs to components · **Routes/Route** define pages, **Link/NavLink** navigate · **Outlet** shares layouts · **useParams** reads `:id`, **useSearchParams** reads `?query`, **useNavigate** moves from code.